

Sistemas Operacionais

3 – Sincronização e Comunicação entre Processos

Prof. Rafael Milbradt

Contato: rmilbradt@gmail.com

Ementa

- UNIDADE 3 – SINCRONIZAÇÃO E COMUNICAÇÃO ENTRE PROCESSOS
 - 3.1 – Introdução / Conceituação
 - 3.2 – Exclusão Mútua
 - 3.3 – Soluções de Hardware
 - 3.4 – Soluções de Software
 - 3.5 – Semáforos
 - 3.6 – Monitores
 - 3.7 – Problemas clássicos
 - 3.8 – Deadlock

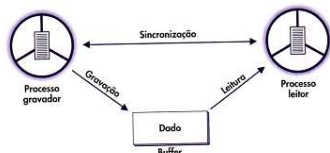
09/04/2026

Prof. Rafael

2

Sinc. e Comunic. entre Processos

- Aplicação concorrente:
 - Necessidade de comunicação entre processos/threads através de variáveis compartilhadas na MP ou troca de mensagens;
 - SO precisa dispor de mecanismos de controle;



Processo leitor: lê apenas se o buffer tem algum dado;

Processo gravador: grava apenas se o buffer tem espaço;

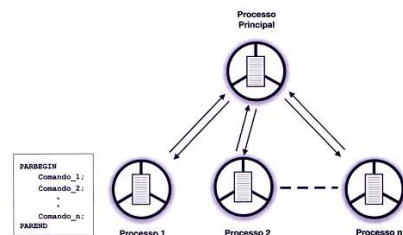
09/04/2026

Prof. Rafael

3

Sinc. e Comunic. entre Processos

- Especificação de Concorrência em Programas:
 - Comandos: PARBEGIN e PAREND (Dijkstra, 1965)



09/04/2026

Prof. Rafael

4

Sinc. e Comunic. entre Processos

- Problemas de Compartilhamento de Recursos:
 - Suponha uma rotina de atualização de Saldo Bancário:

```
PROGRAM Conta_Corrente;
<
  READ (Arg_Contas, Reg_Cliente);
  READLN (Valor_Dep_Ret);
  Reg_Cliente.Saldo := Reg_Cliente.Saldo + Valor_Dep_Ret;
  WRITE (Arg_Contas, Reg_Cliente);
  >
END.
```

- Suponha uma conta possui 1000 reais de saldo e que dois caixas vão fazer operações simultâneas:

- Caixa 1: retirada de 200 reais;
- Caixa 2: depósito de 300 reais;

Tabela 7.1 Problema de Concorrência I

Caixa	Instrução	Saldo arquivo	Valor dep/ret	Saldo memó
1	READ	1.000	*	1.000
1	READLN	1.000	-200	1.000
1	+	1.000	-200	800
2	READ	1.000	*	1.000
2	READLN	1.000	+300	1.000
2	+	1.000	+300	1.300
1	WRITE	800	-200	800
2	WRITE	1.300	+300	1.300

09/04/2026

Prof. Rafael

5

Sinc. e Comunic. entre Processos

- Outra versão do mesmo problema:

Processo A	Processo B	Processo A	Processo B
$X := X + 1;$	$X := X - 1;$	LOAD X, R_0	LOAD X, R_0
		ADD $1, R_0$	SUB $1, R_0$
		STORE R_0, X	STORE R_0, X

Tabela 7.2 Problema de Concorrência II

Processo	Instrução	X	R_0	R_0
A	LOAD X, R_0	2	2	*
A	ADD $1, R_0$	2	3	*
B	LOAD X, R_0	2	*	2
B	SUB $1, R_0$	2	*	1
A	STORE R_0, X	3	3	*
B	STORE R_0, X	1	*	1

09/04/2026

Prof. Rafael

6

Sinc. e Comunic. entre Processos

• Exclusão Mútua:

- A solução mais simples para evitar os problemas anteriores é impedir que dois processos acessem o mesmo recurso simultaneamente;
- Região Crítica: parte do código onde se faz acesso ao recurso compartilhado;
- Protocolos de acesso a região crítica:

```
BEGIN
.
.
  Entra_Regiao_Critica; (* Protocolo de Entrada *)
  Regiao_Critica;
  Sai_Regiao_Critica; (* Protocolo de Saída *)
.
.
END;
```

09/04/2026

Prof. Rafael

7

7

Sinc. e Comunic. entre Processos

• Problema de Sincronismo 1:

- Região crítica: acesso ao saldo da conta e alteração do saldo da conta;

• Problema de Sincronismo 2:

- Região crítica: acesso e alteração da variável x;

• Exclusão mútua: impede acesso simultâneo à região crítica;

- *Starvation* ou espera indefinida: processo que nunca consegue acessar a sua região crítica;
 - SO é responsável por escolher qual dos processos em espera vai ter acesso a região crítica;
 - Dependendo do método de escolha pode ocorrer a espera indefinida;
- Processo fora da região crítica mantém a alocação do recurso e impede que outros processos entrem na zona crítica.

09/04/2026

Prof. Rafael

8

8

Sinc. e Comunic. entre Processos

• Soluções em Hardware:

- Desabilitar interrupções:
 - Compromete a multiprogramação;
 - Usado em algumas operações muito críticas dentro do kernel.

```
BEGIN
.
.
  Desabilita_interrupcoes;
  Regiao_critica;
  Habilita_interrupcoes;
.
.
END;
```

09/04/2026

Prof. Rafael

9

9

Sinc. e Comunic. entre Processos

• Instrução test-and-set:

- Test-and-set (x, y):

- Pega o valor lógico de y e copia para x e atribui verdadeiro a y;
- Y: variável lógica de bloqueio que indica se a região crítica está bloqueada ou não.
 - Se x estiver true já estava bloqueado;
 - Se x estiver false, pode entrar e o bloqueio de y já ocorreu;
- Instrução especial presente em alguns processadores;
- Uma instrução é indivisível e não pode ser interrompida;
 - Interrupções aguardam a execução da instrução corrente;

09/04/2026

Prof. Rafael

10

10

Sinc. e Comunic. entre Processos

```
PROGRAM Test_and_Set;
  VAR Bloqueio : BOOLEAN;

  PROCEDURE Processo_A;
  VAR Pode_A : BOOLEAN;
  BEGIN
    REPEAT
      Pode_A := true;
    WHILE (Pode_A) DO
      Test_and_Set (Pode_A, Bloqueio);
      Regiao_Critica_A;
      Bloqueio := false;
    UNTIL false;
  END;

  PROCEDURE Processo_B;
  VAR Pode_B : BOOLEAN;
  BEGIN
    REPEAT
      Pode_B := true;
    WHILE (Pode_B) DO
      Test_and_Set (Pode_B, Bloqueio);
      Regiao_Critica_B;
      Bloqueio := false;
    UNTIL false;
  END;

  BEGIN
    Bloqueio := false;
    PARBEGIN
      Processo_A;
      Processo_B;
    PAREND;
  END;
```

Vantagens: grande simplicidade de implementação

Desvantagens:

- Espera ocupada;
- Escolha do próximo é arbitrária, possibilitando starvation;

09/04/2026

Prof. Rafael

11

11

Sinc. e Comunic. entre Processos

• Soluções por software:

- Primeiro algoritmo:

Implementa a exclusão mútua porém possui limitações:

```
PROGRAM Algoritmo_1;
  VAR Vez : CHAR;

  PROCEDURE Processo_A;
  BEGIN
    REPEAT
      WHILE (Vez = 'B') DO (* Não faz nada *);
      Regiao_Critica_A;
      Vez := 'A';
      Processamento_A;
    UNTIL false;
  END;

  PROCEDURE Processo_B;
  BEGIN
    REPEAT
      WHILE (Vez = 'A') DO (* Não faz nada *);
      Regiao_Critica_B;
      Vez := 'B';
      Processamento_B;
    UNTIL false;
  END;

  BEGIN
    Vez := 'A';
    PARBEGIN
      Processo_A;
      Processo_B;
    PAREND;
  END;
```

- Suporta apenas 2 processos alternadamente;
- Se um processo precisa acessar a região crítica com maior frequência sempre vai precisar esperar pelo outro, mesmo que a região crítica não esteja sendo usada.

09/04/2026

Prof. Rafael

12

12

Sinc. e Comunic. entre Processos

• Segundo algoritmo:

```
PROGRAM Algoritmo_2;
VAR CA, CB : BOOLEAN;
BEGIN
  PROCEDURE Processo_A;
  BEGIN
    REPEAT
      WHILE (CB) DO (* Nao faz nada *);
      CA := true;
      Regiao_Critica_A;
      CA := false;
      Processamento_A;
    UNTIL false;
  END;
  PROCEDURE Processo_B;
  BEGIN
    REPEAT
      WHILE (CA) DO (* Nao faz nada *);
      CB := true;
      Regiao_Critica_B;
      CB := false;
      Processamento_B;
    UNTIL false;
  END;
  BEGIN
    CA := false;
    CB := false;
    PARBEGIN
      Processo_A;
      Processo_B;
    PAREND;
  END;
END.
```

Prof. Rafael

13

13

Sinc. e Comunic. entre Processos

• Terceiro algoritmo:

```
PROGRAM Algoritmo_3;
VAR CA, CB : BOOLEAN;
BEGIN
  PROCEDURE Processo_A;
  BEGIN
    REPEAT
      CA := true;
      WHILE (CB) DO (* Nao faz nada *);
      Regiao_Critica_A;
      CA := false;
      Processamento_A;
    UNTIL false;
  END;
  PROCEDURE Processo_B;
  BEGIN
    REPEAT
      CB := true;
      WHILE (CA) DO (* Nao faz nada *);
      Regiao_Critica_B;
      CB := false;
      Processamento_B;
    UNTIL false;
  END;
  BEGIN
    CA := false;
    CB := false;
    PARBEGIN
      Processo_A;
      Processo_B;
    PAREND;
  END;
END.
```

09/04/2026

Prof. Rafael

14

Garante a exclusão mútua porém cria outro problema:

Se os dois processos alterarem CA e CB simultaneamente, nenhum dos dois entrará na região crítica.

14

Sinc. e Comunic. entre Processos

• Quarto Algoritmo:

- Parecido com o alg. 3;
- Bloqueia a variável de estado antes de entrar na região crítica, porém existe a possibilidade de reverter;
- Garante a exclusão mútua e não gera bloqueio;
- Gera outras situações indesejadas como os dois processos esperando sem nenhum estar na RC, mesmo que temporariamente;

```
PROGRAM Algoritmo_4;
VAR CA, CB : BOOLEAN;
BEGIN
  PROCEDURE Processo_A;
  BEGIN
    REPEAT
      CA := true;
      WHILE (CB) DO
        BEGIN
          CA := false;
          { pequeno intervalo de tempo aleatório }
          CA := true;
        END;
      Regiao_Critica_A;
      CA := false;
    UNTIL false;
  END;
  PROCEDURE Processo_B;
  BEGIN
    REPEAT
      CB := true;
      WHILE (CA) DO
        BEGIN
          CB := false;
          { pequeno intervalo de tempo aleatório }
          CB := true;
        END;
      Regiao_Critica_B;
      CB := false;
    UNTIL false;
  END;
  BEGIN
    CA := false;
    CB := false;
    PARBEGIN
      Processo_A;
      Processo_B;
    PAREND;
  END;
END.
```

09/04/2026

Prof. Rafael

15

Sinc. e Comunic. entre Processos

• Algoritmo de Dekker:

- Primeira solução de software que garantiu a exclusão mútua, sem outros problemas;
- Baseado nos algoritmos 1 e 4;
- Lógica complexa;

09/04/2026

Prof. Rafael

16

16

Sinc. e Comunic. entre Processos

• Algoritmo de Peterson:

- Processo sinaliza a intenção de entrar através das variáveis de condição (CA e CB) porém cede a vez a outro processo;
- Garante exclusão mútua sem possibilidade de *starvation*;
- Pode ser adaptado para N processos;
- Principal fraqueza: espera ocupada;

```
PROGRAM Algoritmo_Peterson;
VAR CA, CB : BOOLEAN;
    Vez : CHAR;
BEGIN
  PROCEDURE Processo_A;
  BEGIN
    REPEAT
      CA := true;
      Vez := 'A';
      WHILE (CB and Vez = 'B') DO (* Nao faz nada *);
      Regiao_Critica_A;
      CA := false;
      Processamento_A;
    UNTIL false;
  END;
  PROCEDURE Processo_B;
  BEGIN
    REPEAT
      CB := true;
      Vez := 'B';
      WHILE (CA and Vez = 'A') DO (* Nao faz nada *);
      Regiao_Critica_B;
      CB := false;
      Processamento_B;
    UNTIL false;
  END;
  BEGIN
    CA := false;
    CB := false;
    PARBEGIN
      Processo_A;
      Processo_B;
    PAREND;
  END;
END.
```

09/04/2026

Prof. Rafael

17

17

Sinc. e Comunic. entre Processos

• Sincronização condicional:

- Situação em que o acesso compartilhado exige uma condição de acesso;
- Por exemplo problema do buffer limitado ou dos produtores/consumidores;
- Grava_Buffer e Le_Buffer implementam exclusão mútua;
- Existe o Problema da espera ocupada;

```
PROGRAM Produtor_Consumidor_1;
CONST TamBuf = (* Tamanho qualquer *);
TYPE Tipo_Dado = (* Tipo qualquer *);
VAR Buffer : ARRAY [1..TamBuf] OF Tipo_Dado;
    Dado_1 : Tipo_Dado;
    Dado_2 : Tipo_Dado;
    Cont : 0..TamBuf;
BEGIN
  PROCEDURE Produtor;
  BEGIN
    REPEAT
      Produz_Dado (Dado_1);
      WHILE (Cont = TamBuf) DO (* Nao faz nada *);
      Grava_Buffer (Dado_1, Buffer);
      Cont := Cont + 1;
    UNTIL false;
  END;
  PROCEDURE Consumidor;
  BEGIN
    REPEAT
      WHILE (Cont = 0) DO (* Nao faz nada *);
      Le_Buffer (Dado_2, Buffer);
      Consume_Dado (Dado_2);
      Cont := Cont - 1;
    UNTIL false;
  END;
  BEGIN
    Cont := 0;
    PARBEGIN
      Produtor;
      Consumidor;
    PAREND;
  END;
END.
```

09/04/2026

Prof. Rafael

18

18

Sinc. e Comunic. entre Processos

- Semáforos:
 - Proposto por Dijkstra em 1965;
 - Implementação simples de exclusão mútua e sincronização condicional, sem espera ocupada;
 - Implementado na maioria das linguagens e SOs atuais;
 - Semáforo: valor inteiro sem sinal;
 - UP: operação indivisível de incremento no valor do semáforo;
 - DOWN: operação indivisível de decremento no valor do semáforo;
 - DOWN em um semáforo com o valor 0 causa um bloqueio no processo até que outro faça um UP;
 - MUTEX: semáforo apenas para exclusão mútua que assume apenas os valores 0 ou 1;

09/04/2026

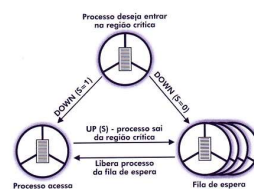
Prof. Rafael

19

19

Sinc. e Comunic. entre Processos

- Exclusão Mútua usando semáforos:



```

PROGRAM Semaforo_1;
VAR S : Semaforo := 1; (* inicializacao do semaforo *)
PROCEDURE Processo_A;
BEGIN
  REPEAT
    DOWN (S);
  UNTIL false;
END;
PROCEDURE Processo_B;
BEGIN
  REPEAT
    DOWN (S);
  UNTIL false;
END;
PARBEGIN
  Processo_A;
  Processo_B;
PAREND;
END.

```

09/04/2026

Prof. Rafael

20

20

Sinc. e Comunic. entre Processos

- Sincronização condicional usando semáforos:

```

PROGRAM Produtor_Consumidor_2;
CONST TamBuf = 2;
TYPE Tipo_Dado = (* Tipo qualquer *);
VAR Vazio : Semaforo := TamBuf;
    Cheio : Semaforo := 0;
    Mutex : Semaforo := 1;
    Buffer : ARRAY [1..TamBuf] OF Tipo_Dado;
    Dado_1 : Tipo_Dado;
    Dado_2 : Tipo_Dado;
PROCEDURE Produtor;
BEGIN
  REPEAT
    Produz_Dado (Dado_1);
    DOWN (Vazio);
    DOWN (Mutex);
    Grava_Buffer (Dado_1, Buffer);
    UP (Mutex);
    UP (Cheio);
  UNTIL false;
END;
PROCEDURE Consumidor;
BEGIN
  REPEAT
    DOWN (Cheio);
    DOWN (Mutex);
    Le_Buffer (Dado_2, Buffer);
    UP (Mutex);
    UP (Vazio);
    Consome_Dado (Dado_2);
  UNTIL false;
END;
PARBEGIN
  Produtor;
  Consumidor;
PAREND;
END.

```

09/04/2026

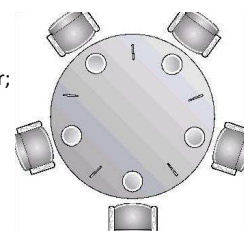
Prof. Rafael

21

21

Sinc. e Comunic. entre Processos

- Problema dos Filósofos Jantando:
 - Problema de sincronização clássico proposto por Dijkstra;
 - Cinco pratos e cinco garfos:
 - 5 filósofos: sentar, comer e pensar;
 - Comer: 2 garfos;
 - Se todos os filósofos segurarem apenas 1 garfo, nenhum come;



09/04/2026

Prof. Rafael

22

22

Sinc. e Comunic. entre Processos

- Formas de resolver o problema de bloqueio:

- Permitir apenas 4 filósofos sentados;
- Permitir que um filósofo pegue um garfo apenas se o outro estiver disponível;
- Filósofos ímpares → garfo da direita, pares → garfo da esquerda;

```

PROGRAM Filosofo_3;
VAR Garfos : ARRAY [0..4] of Semaforo := 1;
    I : INTEGER;
PROCEDURE Filosofo (I : INTEGER);
BEGIN
  REPEAT
    Pensando;
    DOWN (Garfos[I]);
    DOWN (Garfos[(I+1) MOD 5]);
    Comendo;
    UP (Garfos[I]);
    UP (Garfos[(I+1) MOD 5]);
  UNTIL false;
END;
PARBEGIN
  FOR I := 0 TO 4 DO
    Filosofo (I);
  PAREND;
END.

```

09/04/2026

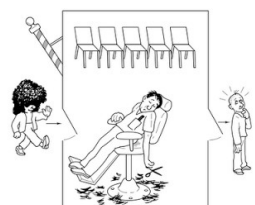
Prof. Rafael

23

23

Sinc. e Comunic. entre Processos

- Na barbearia há uma cadeira de barbeiro e cinco cadeiras para clientes;
- Quando um cliente chega, se o barbeiro estiver trabalhando, ele senta se houver cadeira vazia ou vai embora;
- Se o barbeiro fica sem clientes ele senta na cadeira do barbeiro e dorme.



09/04/2026

Prof. Rafael

24

24

Sinc. e Comunic. entre Processos

- Clientes: semáforo para clientes;
- Barbeiro: semáforo para indicar se o barbeiro está trabalhando ou não;
- Espera: quantidade de clientes que estão esperando;
- Mutex: exclusão mútua da variável espera;

```

PROGRAM Barbeiro;
CONST Cadeiras = 5;
VAR Clientes : Semaforo := 0;
    Barbeiro : Semaforo := 0;
    Mutex : Semaforo := 1;
    Espera : integer := 0;

PROCEDURE Barbeiro;
BEGIN
    REPEAT
        DOWN (Clientes);
        DOWN (Mutex);
        Espera := Espera - 1;
        UP (Barbeiro);
        UP (Mutex);
        Corta_Cabelo;
    UNTIL false;
END;

PROCEDURE Cliente;
BEGIN
    DOWN (Mutex);
    IF (Espera < Cadeiras)
    BEGIN
        Espera := Espera + 1;
        UP (Clientes);
        UP (Mutex);
        DOWN (Barbeiro);
        Ter_Cabelo_Cortado;
    END
    ELSE UP (Mutex);
END;

```

09/04/2026

Prof. Raf

25

25

Sinc. e Comunic. entre Processos

- Monitores:
 - Semáforos em alto nível;
 - Implementados pelo compilador;
 - Mais simples de usar e levam a menos erros pois sua aplicação é estruturada;
 - Maioria das linguagens atuais disponibiliza monitores;

09/04/2026

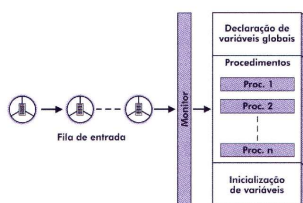
Prof. Rafael

26

26

Sinc. e Comunic. entre Processos

- O monitor organiza uma fila de entrada aos procedimentos em região crítica;
- Possui variáveis globais visíveis apenas no seu escopo e rotinas de inicialização destas;



09/04/2026

Prof. Rafael

27

27

Sinc. e Comunic. entre Processos

- Exclusão Mútua com monitores:
 - Não é realizada diretamente pelo programador;
 - Regiões críticas são implementadas dentro do monitor;
 - Variáveis compartilhadas são implementadas como variáveis globais do monitor.

```

PROGRAM Monitor_1;
    MONITOR Regiao_Critica;
    VAR X : INTEGER;

    PROCEDURE Soma;
    BEGIN
        X := X + 1;
    END;

    PROCEDURE Diminui;
    BEGIN
        X := X - 1;
    END;

    BEGIN
        X := 0;
    END;

    BBEGIN
        PARBEGIN
            Regiao_Critica.Soma;
            Regiao_Critica.Diminui;
        PAREND;
    ENDBEGIN
    END;

```

09/04/2026

Prof. Rafael

28

28

Sinc. e Comunic. entre Processos

- Sincronização Condicional utilizando monitores:
 - Pode ser implementada através de variáveis especiais de condição;
 - As variáveis de condição são manipuladas por duas funções:
 - WAIT: coloca o processo em espera até que algum processo sinalize alteração na variável de condição;
 - SIGNAL: processo sinaliza alteração na variável de estado.
 - Caso não existam processos esperando, não surte efeito nenhum;
 - Libera apenas um processo na fila de espera da variável de condição;
 - Qualquer processo pode executar procedimento de um monitor mesmo quando outros processos estão aguardando;
 - WAIT não bloqueia a região crítica, apenas o processo.

09/04/2026

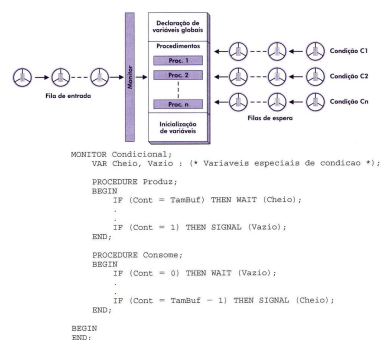
Prof. Rafael

29

29

Sinc. e Comunic. entre Processos

- Sincronização condicional utilizando monitores:



09/04/2026

Prof. Rafael

30

30